

Response to the Review Team for OPRE-2025-04-1885

“Optimizing LLM Inference: Fluid-Guided Online Scheduling with Memory Constraints”

We thank the Area Editor, the Associate Editor, and the three referees for their careful and constructive feedback. The reports made clear that the paper addresses a timely problem but that the original manuscript did not present the model, the queueing interpretation, the memory mechanism, and the numerical evidence at the level of clarity expected for *Operations Research*. We took this concern seriously and have substantially rewritten the manuscript rather than treating the revision as a list of local corrections.

Before responding point by point, we summarize the main changes.

- **Reframed the objective and queueing interpretation.** We revised Sections 2–4 to state the open-system interpretation more carefully. In a stable open system, completed requests match the offered load; the relevant operational question is which arrival rates a policy can stabilize while avoiding eviction-induced restarts and maintaining bounded delay. Accordingly, the paper now uses *effective throughput* for completed service net of evictions, interprets the fluid model as a stability-region benchmark, and reports latency as a function of the arrival rate in Section 6.
- **Rewrote the model and notation.** Section 2 now introduces prefill, decode, batching, KV-cache growth, the memory constraint, and the policy space narratively. The main notation is collected in the Notation Summary appendix, and the stage notation is separated from prompt lengths and output lengths. We also clarified that the memory constraint applies to the KV caches of all in-system requests retained on the GPU, not merely to requests in the current batch.
- **Clarified the time model and its scope.** We expanded the discussion around Equation (1), explaining that the affine iteration-time model is intended for decode-dominant serving regimes where KV-cache loading is the main marginal cost. We added direct A100 validation in the Additional Experiments appendix and reported real-GPU SGLang experiments. We also explain why the model is particularly well matched to prefill-decode disaggregated systems, where the decode server is exactly the regime captured by our linear approximation.
- **Clarified the theoretical contribution.** We removed the misleading “heavy traffic” terminology from the narrative and now describe the guarantees as long-horizon/asymptotic guarantees under fixed load. The main text now explains why the problem is not a standard queueing exercise: KV-cache memory grows with decode progress, evictions can restart partially completed work, and unknown output lengths reveal type information only through the service trajectory. The WAIT and Nested WAIT thresholds reduce this high-dimensional state-action space to tractable boundary queues.
- **Expanded and reorganized the experiments.** Section 6 now reports latency and effective-completion-rate curves over arrival-rate sweeps, including underloaded, near-capacity, and overloaded regimes. We added or revised validation for simulator fidelity, repeated-run variability near the stability boundary, long-decode workloads, threshold-without-waiting, real-GPU single-type and real-data experiments, and prefill-decode-disaggregated deployment.
- **Updated positioning and scope.** The Introduction and related-work discussion now clarify that our contribution is a modeling and scheduling contribution for memory-constrained LLM inference, not a

claim of a new heavy-traffic limit. We also added discussion of concurrent theoretical work and prefill-decode disaggregated systems, where the decode-only setting makes the linear iteration-time model especially natural.

- **Professionalized the exposition.** We rewrote large portions of Sections 1–6 and the appendices for clarity, removed ambiguous terminology, added mechanism-level explanations after algorithms and theorems, and modularized the appendix proofs.

Main concern	How the revised manuscript addresses it
Open-system throughput	Recasts the objective around stability-region expansion, effective throughput net of evictions, latency, and TTFT.
Memory realism	Counts retained KV caches of admitted requests, explains eviction-induced restarts, and studies near-capacity and long-decode regimes.
Linear time model	States the decode-dominant scope, validates the model against A100 measurements, and discusses PD-disaggregated deployments.
Theoretical novelty	Positions WAIT/Nested WAIT as fluid-guided threshold policies that reduce a high-dimensional memory-growth control problem to tractable boundary queues.
Writing quality	Rewrites Sections 1–6, adds notation summary and examples, and reorganizes the appendices around theorem-specific proof logic.

Response to the Area Editor

Comment. *The paper requires a complete expositional overhaul; the original presentation had notation inconsistencies, insufficient intuition, and proof summaries that were too high level to be useful.*

Our response. We agree. The revised manuscript has been reorganized so that the reader first sees the operational mechanism, then the mathematical abstraction, and only then the policy and analysis. Section 2 now introduces notation as it enters the model, with a consolidated notation table moved to the Notation Summary appendix. Sections 4 and 5 now include examples and mechanism-level explanations immediately before the algorithms, and the theorem discussions explain what the memory conditions mean in terms of batching, KV-cache growth, and boundary queues. The appendices have also been modularized so that each theorem proof has its own proof section and the main text explains the proof logic before deferring details.

Changes in the revised manuscript.

- Rewrote Section 2 around prompts, inference stages, batching, memory, and the objective.
- Added a notation summary in the Notation Summary appendix.
- Added examples and explanatory figures for WAIT and Nested WAIT in Sections 4 and 5.
- Reorganized the proof appendix into modular sections for Propositions and Theorems 1–3.
- Added theorem-interpretation paragraphs that connect each guarantee to the primitive queueing and memory mechanisms.

Comment. *The model assumptions need either modification or better justification, especially the affine iteration-time model and the treatment of memory, OOM, and preemption.*

Our response. We retained the simple model but made its scope explicit. The affine iteration-time model is now presented as a decode-dominant approximation: it separates the fixed per-iteration overhead from the marginal cost of serving active KV caches. We also state that the approximation is especially accurate in prefill-decode disaggregated deployments, where the decode server processes decode-only work. To avoid relying only on simulation, the Additional Experiments appendix compares Vidur predictions with direct A100 measurements over the batch-size range used in the experiments. The memory model has also been clarified: admitted requests that remain active keep their KV caches on the GPU and count against the memory constraint. If memory is exceeded, the server evicts and restarts an in-progress request, which is counted as lost useful work in the effective-throughput metric.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the affine iteration-time model and its decode-dominant scope.
- Revised Section 2.3 to state that the memory constraint applies to all in-system KV caches retained on the GPU.
- Added real-GPU validation in the Additional Experiments appendix, with A100 measurements and a fitted iteration-time model.
- Added a prefill-decode-disaggregated experiment in the Additional Experiments appendix showing that the same threshold logic applies in a decode-only serving scenario.
- Revised Section 6 and the long-decode eviction table to report eviction-induced restart rates where relevant.

Comment. *The theoretical novelty should be clarified. If the contribution is modeling and problem structuring rather than a new stochastic-process theorem, that should be stated clearly.*

Our response. We revised the framing accordingly. The paper no longer presents the contribution as a heavy-traffic theorem for a standard open queue. Instead, we emphasize the modeling and algorithm-design contribution: LLM inference creates a high-dimensional control problem because memory consumption grows with decode progress, output lengths may be unknown, and eviction can erase partially completed work. WAIT and Nested WAIT impose threshold structure that keeps the system close to the fluid equilibrium and turns the analysis into tractable boundary processes. The mathematics then formalizes why this structured policy prevents memory overflow and achieves the fluid benchmark in the long-horizon regime.

Changes in the revised manuscript.

- Reframed Section 3 as a fluid model and equilibrium analysis rather than a throughput-maximization problem at a fixed arrival rate.
- Rewrote Sections 4.1–4.2 to explain why threshold-based admission reduces the state-action space.
- Rewrote Section 5 to explain how Nested WAIT handles unknown output lengths through segment boundaries and binomial thinning.

- Removed misleading “heavy traffic” language from the main narrative and replaced it with long-horizon/asymptotic language.

Comment. *The paper should better connect the LLM-inference domain to existing stochastic-modeling and scheduling literature.*

Our response. We added related-work discussion to position the paper more precisely. The revised Introduction now explains how our model differs from concurrent theoretical work that abstracts each decoding step as a unit-time operation, while our model retains the batch-composition-dependent iteration time induced by KV-cache loading. We also discuss prefill-decode disaggregated systems, where the decode-side workload aligns closely with the affine iteration-time model. This positioning follows the revision plan’s main framing: the contribution is not only a theorem, but the identification of a memory-growth and eviction-control structure that makes LLM inference amenable to fluid-guided scheduling.

Changes in the revised manuscript.

- Expanded the Other Related Work subsection in the Introduction.
- Added discussion of concurrent LLM-inference scheduling theory and the distinction between unit-time decode abstractions and our batch-dependent time model.
- Added connections to Splitwise and DistServe-style prefill-decode disaggregation.

Response to Reviewer 1

We thank Reviewer 1 for emphasizing the importance of modeling clarity, notation consistency, and OOM safeguards. These comments directly shaped the rewrite of Section 2 and the memory discussion throughout the paper.

R1.1. OOM and KV-cache capacity

Comment. *The reviewer asks how the proposed algorithms ensure that KV-cache usage does not exceed capacity, especially under heavy workloads and uncertain output lengths.*

Our response. We agree that this mechanism was underexplained. The revised manuscript now makes the memory accounting explicit. The relevant memory is the total KV cache retained by all admitted in-system prompts on the GPU. If the system admits too much work, decode progress increases KV-cache usage and may force the serving system to evict and restart a request. The proposed policies are designed to prevent such overflow by controlling the number of prompts that enter and advance through the decode stages. For known output lengths, WAIT uses type-specific thresholds derived from the fluid equilibrium. For unknown output lengths, Nested WAIT adds segment-level thresholds and safety buffers so that prompts that survive to later decode segments do not exceed the memory budget with high probability.

Changes in the revised manuscript.

- Added the memory-growth example and memory-constraint discussion in Section 2.3.
- Added Example 2 in Section 4 to show how FCFS can trigger eviction cascades even when the nominal capacity is sufficient.
- Added Example 3 and the Nested WAIT explanation in Section 5 to show how unknown output lengths are handled through segment boundaries.
- Added a long-decode near-capacity eviction diagnostic in Section 6, where Sarathi incurs a positive eviction-induced restart rate while WAIT avoids eviction.

R1.2. Linearity of Equation (1)

Comment. *The reviewer asks for additional justification of the affine iteration-time assumption and for discussion of possible nonlinear relationships across GPU architectures.*

Our response. We have expanded the discussion of Equation (1) and clarified its intended operating regime. The affine form is a parsimonious approximation for decode-dominant serving, where prompts spend most iterations in decode and the marginal cost is driven by active KV-cache length. We do not claim that this model captures every possible hardware and workload regime. Long-prefill workloads and architectures with different memory-access patterns may require richer time models. The revised manuscript states this limitation and adds two forms of empirical support: local validation of the iteration-time model and direct A100 validation against SGLang.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the fixed-overhead and marginal-KV-cache components of Equation (1).
- Added Additional Experiments appendix validation comparing Vidur predictions with direct A100 measurements.
- Added discussion in the Introduction and Additional Experiments appendix that prefill-decode disaggregation is a natural setting where the decode-side linear approximation is especially accurate.

R1.3. Notation inconsistencies

Comment. *The reviewer notes that the original manuscript used similar notation for decode stage, prompt length, output length, and current processing stage.*

Our response. We agree. The revised manuscript separates these concepts. Decode stage is denoted by the stage index s . Prompt length, output length, and type-dependent lengths use separate notation. To reduce the burden on the reader, we moved the full notation table to the Notation Summary appendix and introduce symbols narratively as they first appear in the main text.

Changes in the revised manuscript.

- Revised Section 2 to introduce stage notation consistently.

- Added the Notation Summary appendix table collecting the main symbols used throughout the paper.
- Updated the algorithms and theorem statements to use the same stage and threshold notation.

R1.4. Policy space and batching operations

Comment. *The reviewer asks for a clearer mathematical characterization of the policy space Π and the batching operations implied by a policy.*

Our response. We added a dedicated objective and policy-space subsection. The revised text explains that a policy observes the current queues and KV-cache state, chooses which prompts to admit or advance, and must respect the information available at the decision epoch. This makes the algorithms easier to interpret as concrete elements of the policy class rather than standalone procedures introduced after the model.

Changes in the revised manuscript.

- Revised Section 2.4 to define the objective, the policy class, and the main performance metrics.
- Clarified the relation between policy decisions, batch formation, memory consumption, latency, TTFT, and effective throughput.

R1.5. Need for intuition after definitions and algorithms

Comment. *The reviewer asks for more explanatory text after key definitions and Algorithm 1.*

Our response. We added intuition throughout the main text. Section 2 now explains each modeling component through the running LLM-inference example. Section 4 begins with a concrete FCFS failure example before introducing WAIT. Section 5 begins with an unknown-output-length example before introducing Nested WAIT. The algorithm descriptions now explain why thresholds regulate both admission and downstream decode-stage composition.

Changes in the revised manuscript.

- Added explanatory text around the introductory inference figure and the batching example figure.
- Added Example 2 before WAIT and Example 3 before Nested WAIT.
- Added mechanism paragraphs after Algorithm 1 and Algorithm 2.

R1.6. Typos

Comment. *The reviewer notes several typos, including d versus d_1 and “Throughput” versus “Throughput.”*

Our response. These have been corrected. We also conducted a full consistency sweep for notation, terminology, cross-references, and repeated AI-style phrasing.

Changes in the revised manuscript.

- Corrected the throughput macro.
- Corrected d , d_0 , and d_1 inconsistencies in the model and fluid sections.
- Removed duplicate or inconsistent notation flagged in the reviewer comments.

Response to Reviewer 2

We thank Reviewer 2 for the detailed and constructive technical report. The comments on the time model, the role of the memory assumption, the proof structure, and experimental reproducibility were particularly helpful.

R2.1. Scope of the affine iteration-time model

Comment. *The reviewer suggests that Equation (1) may omit prefill attention, linear layers, and decode attention terms, and asks which asymptotic regime the paper targets.*

Our response. This is exactly the right way to interpret the modeling choice. The revised manuscript now states that Equation (1) targets the decode-dominant regime in which the marginal time cost is primarily driven by active KV-cache length. We agree that long-context prefill and other regimes may require additional terms. Rather than claiming universality, we now position Equation (1) as a transparent approximation that captures the admission-control trade-off central to the paper: admitting more prompts increases batching efficiency but also raises iteration time and memory consumption through larger KV caches. We also added A100 validation over the relevant batch-size range.

Changes in the revised manuscript.

- Revised Section 2.2 to explain the operating regime of Equation (1).
- Added the simulator-calibration figure in the Additional Experiments appendix, comparing Vidur iteration-time predictions with direct A100 measurements.
- Added discussion of prefill-decode disaggregation, where the decode server removes the prefill-attention terms and the linear KV-cache approximation is especially appropriate.

R2.2. Clarifications in Section 2

Comment. *The reviewer provides several minor comments on notation, prefill throughput, the policy class, and whether preemption is permitted.*

Our response. We revised Section 2 to address these points. The stage notation has been cleaned up, the role of prefill and decode is described explicitly, and the objective discussion now clarifies the metrics studied in the paper. The revised model does not rely on costless swapping of inactive KV caches. Requests that are admitted and remain active retain their KV caches on the GPU and count toward memory consumption; if memory is exceeded, the serving system evicts and restarts work, which is treated as lost useful service.

Changes in the revised manuscript.

- Rewrote Section 2.1–2.4 for notation and policy clarity.
- Added the Notation Summary appendix.
- Clarified that the memory constraint applies to retained KV caches of in-system requests.

R2.3. The assumption $C \geq M^*$

Comment. *The reviewer notes that $C \geq M^*$ may be a nontrivial requirement, especially for very long outputs, and asks whether the experiments satisfy or violate this condition.*

Our response. We agree that this assumption should not be presented as benign. The revised paper treats M^* as the ideal-fluid memory requirement for stability, not as a universally attainable capacity level. The theory characterizes what is possible when the system has enough memory to support the fluid operating point. The experiments deliberately include regimes near and beyond the stability boundary, including long-decode workloads, to show what happens when memory pressure becomes severe. In those regimes, WAIT remains useful because it controls admission and prevents eviction-induced restarts even when the system cannot simply rely on excess memory.

Changes in the revised manuscript.

- Revised the discussion around M^* in Sections 3 and 4.
- Added long-decode experiments in Section 6, including p512d1000.
- Added the prefill-decode-disaggregated deployment subsection in the Additional Experiments appendix, where decode-side memory pressure is isolated.

R2.3a. Segment-wise design for unknown or long output lengths

Comment. *The reviewer notes that maintaining a separate stage for every possible output length may be impractical in long-output settings.*

Our response. We agree. The revised paper separates the clean theorem statement from the implementable segment design. The main theorem presents the analytically transparent case in which output lengths map to segment boundaries. The Extensions appendix then discusses general segment designs that group nearby output lengths into fewer segments. This is the form used in the real-data experiments: segment-level thresholds preserve the same continuation-probability logic while avoiding the need to maintain a separate threshold for every individual decode length.

Changes in the revised manuscript.

- Revised Section 5 to explain segment boundaries as the points where output-length information is revealed.
- Added Extensions appendix discussion of general segment design and coarser segmentations.

- Revised Section 6.2 to describe how empirical continuation probabilities determine the segment thresholds in the real-data workload.

R2.4. Proof of Theorem 1 and decode-stage dynamics

Comment. *The reviewer asks how the proof accounts for l' decode stages and whether completions occur only after enough batch executions.*

Our response. We have clarified the reduction used in the proof. WAIT is designed so that once a type is scheduled, threshold-sized groups advance together through the decode stages. This threshold structure is what permits the proof to reduce the multi-stage process to a boundary queue for each type. The main text now explains this dimensionality reduction before the theorem discussion, and the appendix proof explicitly separates the batch index from the decode-stage index.

Changes in the revised manuscript.

- Revised Section 4.2 to explain how threshold-sized batches move through decode stages.
- Revised the proof appendix for Theorem 1 to define the batch index separately from the stage index.
- Added explanatory text connecting queue accumulation to throughput loss in the proof of Theorem 1.

R2.5. Thresholds without waiting

Comment. *The reviewer asks what happens if the algorithm uses thresholds but does not wait for enough prompts to accumulate.*

Our response. We added a targeted comparison in the Additional Experiments appendix. The threshold-only variant imposes an upper bound on in-system requests but does not wait to form threshold-sized batches. The comparison shows that the waiting component has a regime-dependent role. At low arrival rates, waiting can add modest delay because the server has sufficient capacity to process requests immediately. Near the overload boundary, waiting becomes useful because it regulates when work enters service and improves batch formation. Once the system is overloaded, the aggregate upper bound dominates and the two variants behave more similarly.

Changes in the revised manuscript.

- Added the “Threshold without waiting” subsection in the Additional Experiments appendix.
- Added the wait-versus-threshold-only figure reporting the latency comparison across arrival rates.

R2.6. Role of the asymptotic assumption

Comment. *The reviewer asks what role the original heavy-traffic assumption plays and whether the theorem can be stated without it.*

Our response. We agree that the original terminology was misleading. The analysis does not take a classical heavy-traffic limit in which the load approaches the boundary of the stability region. It studies long-horizon averages under fixed load and a scaled observation horizon. We have removed the heavy-traffic framing from the narrative and now describe the result as an asymptotic or long-horizon guarantee. The assumption is used to compare the stochastic system against the fluid benchmark over a long horizon, not to claim a conventional heavy-traffic diffusion limit.

Changes in the revised manuscript.

- Replaced “heavy traffic” terminology in the main text with asymptotic or long-horizon terminology.
- Revised the theorem discussion in Sections 4 and 5 to state the role of the limit more directly.

R2.7. Formula and proof-detail comments

Comment. *The reviewer lists several formula checks and appendix proof clarifications, including d_0 versus d_1 , M^π versus M^* , undefined notation, and an alternative proof idea.*

Our response. We reviewed these items during the revision. The revised manuscript corrects the d_0/d_1 inconsistencies, clarifies the distinction between M^* and M^π , defines segment probabilities and safety-buffer terms before the Nested WAIT theorem, and reorganizes the proof appendix around the main theorem statements. We also adopted the spirit of the reviewer’s alternative proof suggestion by making the coupling argument more direct and by separating the dominating process from the original queue process.

Changes in the revised manuscript.

- Corrected the d_0/d_1 inconsistencies in Section 3.
- Revised Section 5.2 to define p_k , n_k , M^π , and the safety-buffer interpretation before Theorem 2.
- Revised the proof appendix for theorem-specific proof organization.
- Added citations and references to standard queueing tools where appropriate.

R2.7a. Appendix organization and use of standard stochastic results

Comment. *The reviewer and the Area Editor suggest that the appendix should be easier to verify and should connect more directly to standard stochastic-process tools.*

Our response. We reorganized the proof appendix to make the structure easier to follow. The appendix now separates proposition proofs, the WAIT theorem proof, the Nested WAIT theorem proof, the time-varying extension, and supporting lemmas. Within the theorem proofs, we use standard queueing objects explicitly: a Lindley-type recursion for the threshold queue, random-walk maximum bounds for queue accumulation, and martingale bounds for the Nested WAIT safety buffer. The main text now explains why the policy creates the reduced process before the appendix analyzes it.

Changes in the revised manuscript.

- Split the proof appendix into theorem-specific proof sections.
- Added definitions of the batch index, stage index, and coupled process before using them in proofs.
- Added citations to standard queueing tools where they replace or streamline first-principles derivations.

R2.8. Experimental parameters and reproducibility

Comment. *The reviewer asks for batch-size, memory, capacity, Sarathi configuration, and Nested WAIT parameter details in Section 6.*

Our response. We substantially rewrote Section 6 and the Additional Experiments appendix to make the experimental settings more reproducible. The main text now states the hardware, simulator, baseline configurations, WAIT/Nested WAIT threshold construction, arrival-rate sweeps, and number of replications. Sarathi is described as an FCFS-based policy with chunked prefill, and the memory-reservation rule used by the baselines is stated explicitly. For Nested WAIT, the revised text explains how the total in-flight limit determines the segment thresholds through continuation probabilities.

Changes in the revised manuscript.

- Added hardware and baseline-configuration paragraphs at the start of Section 6.
- Added a WAIT/Nested WAIT settings paragraph in Section 6.
- Revised the real-data subsection to explain the dataset, sampling, segment construction, and threshold allocation.
- Added real-GPU SGLang parameter descriptions, including a parameter table for the real-data GPU run.

R2.9. Simulation fidelity and real-GPU validation

Comment. *The reviewer recommends validating Vidur against actual GPU measurements.*

Our response. We added this validation. The Additional Experiments appendix compares Vidur predictions with direct A100 measurements under SGLang for Llama-2-7B. The validation covers the batch-size range used in the experiments and reports a small mean absolute percentage error and high R^2 . We also added end-to-end real-GPU experiments comparing WAIT with the SGLang baseline scheduler for a single-type workload and for the lmsys-chat-1m prompt distribution.

Changes in the revised manuscript.

- Added the simulator-calibration figure in the real-GPU validation appendix.
- Added end-to-end SGLang comparisons on single-type and real-data workloads.
- Added references to the validation from Section 6.

R2.10. Long outputs

Comment. *The reviewer asks how the algorithm behaves when output length exceeds 1000 tokens.*

Our response. We added a long-decode workload to Section 6. The workload p512d1000 keeps the prefill length fixed and increases the decode length to 1000. This setting stresses KV-cache growth and causes all policies to lose stability at much lower arrival rates. WAIT performs best near the boundary and avoids eviction-induced restarts in the near-capacity diagnostic, illustrating that threshold-based admission remains useful when decode-side memory growth is pronounced.

Changes in the revised manuscript.

- Added the long-decode regime in Section 6.1.
- Added the long-decode figure and near-capacity eviction table for latency, effective completion rate, and eviction-induced restart evidence.

Response to Reviewer 3

We thank Reviewer 3 for the detailed critique. The report identified the most important conceptual weaknesses in the original submission: the open-system throughput interpretation, the misuse of heavy-traffic terminology, the need for meaningful latency experiments, and the need to explain why the theory is not merely a routine positive-recurrence argument. These points led to the largest changes in the revision.

R3.1. Heavy-traffic terminology

Comment. *The reviewer notes that the original “heavy traffic” analysis was not a conventional heavy-traffic limit; it was a long-horizon limit under fixed load.*

Our response. We agree and have corrected the terminology. The revised paper does not present the results as a classical heavy-traffic analysis. Instead, it explains that the theoretical results compare the stochastic system to the fluid benchmark in a long-horizon/asymptotic regime under fixed arrival rates. This change also helped us clarify the role of the theory: the contribution is to structure the LLM-inference control problem through fluid-guided thresholds and to prove that the resulting policy tracks the benchmark, not to derive a new diffusion limit.

Changes in the revised manuscript.

- Replaced heavy-traffic terminology in the main narrative.
- Revised Sections 4 and 5 theorem discussions to state the asymptotic regime more accurately.

R3.2. Throughput in an open system

Comment. *The reviewer points out that in a stable open system, throughput equals the arrival rate, so the relevant question is the stability region rather than throughput at a fixed arrival rate.*

Our response. We agree with this interpretation and revised the paper accordingly. The revised Section 2 objective discussion now explains that a stable system cannot complete more useful work than the offered load. The operational challenge is whether a policy can convert the offered work into completed service while avoiding idle capacity, memory overflow, and eviction-induced restarts. Section 3 uses the fluid model to identify the memory level and stage composition required for stability under the idealized model. Section 6 then reports latency and effective-completion-rate curves over arrival-rate sweeps, so the stability boundary is visible.

Changes in the revised manuscript.

- Revised Section 2.4 to define effective throughput and latency in an open-system way.
- Revised Section 3 to describe the fluid model as a stability-region benchmark.
- Revised Section 6 to explain why stable policies have similar completion rates below the boundary and why latency curves are needed to compare policies within the stable region.

We also changed the terminology around M^* . The revised manuscript no longer describes M^* as a “throughput-optimal memory” level. It is the equilibrium memory level required to sustain the ideal-fluid operating point at the boundary of the stability region. This distinction matters for the open-system interpretation: increasing memory does not raise completed work above the offered load in a stable system, but insufficient memory or poor admission control can shrink the stability region and reduce effective completed service through evictions.

R3.3. Positive recurrence and theoretical significance

Comment. *The reviewer argues that bounded latency under a stable policy may follow from standard positive-recurrence arguments unless the paper explains the nontrivial part of the analysis.*

Our response. This comment was very helpful. We have revised the theory sections to distinguish the standard queueing implication from the nonstandard part of our problem. The difficult step is not that a stable queue has finite delays; it is designing a policy whose high-dimensional LLM-serving state can be controlled under growing memory consumption and eviction risk. WAIT reduces the known-type system to threshold-sized stage flows. Nested WAIT further handles unknown output lengths by using segment boundaries: prompts reveal their type only when they complete at a boundary or survive to the next segment. The coupling analysis exploits this structure to control boundary queues and memory overflow without tracking every prompt across every decode stage.

Changes in the revised manuscript.

- Rewrote Section 4.2 to explain the dimensionality reduction created by WAIT.

- Rewrote Section 5.2 to explain the boundary-queue coupling created by Nested WAIT.
- Revised the theorem interpretation paragraphs to connect memory conditions to overflow prevention rather than simply stating bounded latency.

This revision follows the main point in our revision plan: the policy makes the stochastic analysis simple enough to verify. Without a well-structured threshold policy, the primitive state includes the number of prompts at each decode stage, their retained KV caches, type uncertainty, and the possibility of eviction-induced restarts. The threshold design is what creates the lower-dimensional queueing object analyzed in the proof.

R3.4. Latency experiments and unstable regimes

Comment. *The reviewer notes that latency comparisons in unstable regimes can be misleading and recommends plotting mean latency as a function of arrival rate.*

Our response. We agree and have reworked the numerical section around this diagnostic. The main figures now plot mean end-to-end latency versus arrival rate and pair it with effective completion rate. This presentation shows the underloaded, near-capacity, and overloaded regimes in the same figure. Below the stability boundary, stable policies complete at the offered rate, so latency is the informative comparison. Near the boundary, latency divergence identifies the stability transition. Above the boundary, effective completion rate and eviction-induced restarts show how much useful work each policy can still complete.

Changes in the revised manuscript.

- Added arrival-rate sweeps for single-type, two-type, long-decode, and real-data workloads in Section 6.
- Added finite-horizon validation near the two-type stability boundary.
- Reported each main rate-sweep point as an average over repeated simulation replications.
- Added repeated-run variability analysis in the Additional Experiments appendix.

We also revised the numerical narrative so that latency improvements are not presented only as an overloaded-regime artifact. The synthetic and real-data rate sweeps discuss low-load, near-capacity, and overloaded regimes separately: at low load the policies are often close, while the advantage of WAIT/Nested WAIT becomes more pronounced near capacity because the threshold controls keep the in-system population closer to balance and reduce eviction-induced waste.

R3.5. Algorithmic contribution and latency

Comment. *The reviewer characterizes WAIT as a pipeline-filling idea and asks for a stronger explanation of latency effects and algorithmic contribution.*

Our response. We agree that the pipeline interpretation is a useful starting point, and we now use it to explain the policy more clearly. The revised paper emphasizes what the pipeline view alone does not capture:

KV-cache memory grows with decode progress, admitted jobs can later cause overflow, and evictions restart useful work. WAIT is not only filling a pipeline; it regulates the population at each stage so that arrivals, completions, and memory growth remain balanced. Nested WAIT extends the same principle when output length is unknown, delaying type separation until the prompt’s own decode trajectory reveals whether it is short or long. The numerical section now reports latency over arrival-rate sweeps precisely to show where these controls matter.

Changes in the revised manuscript.

- Added and revised examples in Sections 4 and 5 to explain WAIT and Nested WAIT through concrete system trajectories.
- Revised Section 6 to emphasize latency differences near capacity and stability-boundary expansion.
- Added threshold-without-waiting and real-GPU comparisons in the Additional Experiments appendix.

R3.6. Memory constraint and preempted jobs

Comment. *The reviewer asks whether the memory constraint applies only to jobs in the current batch and whether preserving preempted KV caches elsewhere is realistic.*

Our response. We revised the model to remove this ambiguity. The memory constraint applies to the KV caches of admitted in-system requests retained on the GPU. Requests that are not selected in the current iteration remain in their current queues, and those already in decode retain their KV caches. We do not rely on moving inactive KV caches to CPU memory or SSD at zero cost. If the retained GPU KV-cache memory exceeds capacity, the serving system must evict and restart an in-progress request. This is exactly the loss channel that motivates the admission-control problem studied in the paper.

Changes in the revised manuscript.

- Revised Section 2.3 to state the memory accounting explicitly.
- Revised Algorithm 1 and Algorithm 2 explanations to state that unselected decode prompts retain their KV caches.
- Added eviction-induced restart diagnostics in Section 6.

R3.7. Writing quality, definitions, and interpretation

Comment. *The reviewer notes that the original manuscript was unclear, with incomplete definitions, little interpretation of formulas, and notation inconsistencies.*

Our response. We agree that the original writing did not make the paper’s contribution sufficiently accessible. The revision contains a substantial rewrite of the main text and appendices. The fluid model is now developed more slowly, with each formula followed by its operational interpretation. The algorithms are introduced through examples before pseudocode. The theorem statements are followed by paragraphs explaining what the memory terms mean and how the proof reduces the original system to tractable queues.

Changes in the revised manuscript.

- Rewrote Sections 2–5 for notation, flow, and interpretation.
- Added the notation table in the Notation Summary appendix.
- Added explanatory paragraphs after the fluid-model formulas, algorithms, and theorems.
- Reorganized the appendix proofs and removed several sources of notational ambiguity.

Closing

We appreciate the review team’s detailed guidance. The revised manuscript is substantially different from the original submission: the problem framing is now centered on stability-region expansion, eviction prevention, and latency; the model assumptions are stated more transparently; the theory is positioned as a fluid-guided policy design and dimensionality-reduction argument; and the experiments now directly address the queueing diagnostics requested by the reviewers. We hope that the revised version makes the contribution and limitations of the work much clearer.